

Cantera-TCD Documentation

Cillian Thomas

1 Simplified Transport Properties

Often in Large Eddy Simulation (LES), simplified assumptions are made about mass diffusion to avoid complex multicomponent diffusion calculations. There are also benefits in expressing a Lewis number of a mixture species in order to adapt the stretched response of a flame [1]. This mass diffusion simplification is often coupled with a simplified thermal diffusion model.

A Schmidt number (Sc) for each species can be expressed as:

$$Sc_k = \frac{\mu}{\rho D_k} \quad (1)$$

where k is the species of interest, μ is the dynamic viscosity, and D_k is the mass diffusivity of species k . Mass diffusivity is often modelled using a constant Schmidt number assumption:

$$D_k = \frac{\mu}{\rho Sc_k} \quad (2)$$

and likewise, the mixture diffusivity for a mixture with no preferential mass diffusion:

$$D_{mix} = \frac{\mu}{\rho Sc_{mix}} \quad (3)$$

Thermal diffusivity can be expressed as:

$$\alpha \equiv \frac{\lambda}{\rho c_p} = \frac{\mu}{\rho Pr} \quad (4)$$

where α is the thermal diffusivity, λ is the thermal conductivity, ρ is density, c_p is specific heat. Often thermal diffusion is modelled by a constant Prandtl number. Hence,

$$\lambda = \frac{c_p \mu}{Pr} \quad (5)$$

The Lewis number describes the rate of thermal diffusion to mass diffusion:

$$Le_k = \frac{\lambda}{\rho c_p D_k} = \frac{Sc_k}{Pr} \quad (6)$$

Hence thermal and mass diffusion can be fully defined by constant Lewis (or Schmidt) and Prandtl numbers. For a mixture-defined Lewis number:

$$D_{mix} = \frac{\mu}{\rho Le_{mix} Pr} \quad (7)$$

To implement equation 5 and 7 in Cantera, a copy is made of the `UnityLewisTransport` class. This class is inherited from the `MixTransport` class, and overrides the properties as needed. All methods are defined in the header file, as per the `UnityLewisTransport.h` definition. See `include/cantera/transport/SimplifiedTransport.h` for full implementation.

Equation 5 is implemented in Listing 1.

Listing 1: Implementation of the simplified thermal conductivity.

```
1 double thermalConductivity () override
2 {
3     update_T();
4     update_C();
5 }
```

```

6     m_lambda = m_thermo->cp_mass()
7         * viscosity()/m_prandtl;
8
9     return m_lambda;
10 }

```

Equation 7 is implemented in Listing 2.

Listing 2: Implementation of the simplified mixture diffusivity.

```

1 void getMixDiffCoeffsMass(span<double> d) override
2 {
3     checkArraySize(
4         "SimplifiedTransport::getMixDiffCoeffsMass",
5         d.size(),
6         m_nsp
7     );
8
9     double Dm = thermalConductivity()
10        / (m_thermo->density()
11           * m_thermo->cp_mass()
12           * m_le);
13
14     for (size_t k = 0; k < m_nsp; k++) {
15         d[k] = Dm;
16     }
17 }

```

Default values of $m_prandtl=0.7$ and $m_le=1$ are provided. Setters are provided for user-specified Pr and Le_{mix} , as shown in Listing 3.

Listing 3: Setter methods for the Prandtl and Lewis numbers.

```

1 void setPrandtl(double pr)
2 {
3     this->m_prandtl=pr;
4 }
5
6 void setLe(double le)
7 {
8     this->m_le = le;
9 }

```

The simplified transport model, and the user-specified Pr and Le are set as follows in the Python frontend, as shown in Listing 4.

Listing 4: Python frontend configuration of the simplified transport model.

```

1 gas.transport_model = "simplified"
2 gas.set_lewis_number(user_le)
3 gas.set_prandtl_number(user_pr)

```

In the script `samples/python/stretchedTF/atf_simplified_validation.ipynb`, the simplified transport properties are verified using a freely propagating flame. Mixture-averaged and simplified transport laminar flame speeds are compared in Table 1. The script also validates the temperature profile and laminar flame thickness using both simplified transport and flame thickening.

Table 1: Comparison of the freely propagating laminar flame properties obtained using mixture-averaged and simplified transport. The flame thickness is defined using the maximum temperature-gradient method.

Transport model	Laminar flame speed, s_L^0 m/s	Flame thickness, δ_L^0 mm
Mixture-averaged transport	0.2054	0.5596
Simplified transport	0.2089	0.5643

2 Species, Thermal and Finite Rate Thickening Factors

As documented by Cantera [2], the governing equations for one-dimensional flow are given in Equations 8–11, and are implemented in the `Flow1D` class.

Continuity

$$\frac{\partial(\rho u)}{\partial z} + 2\rho V = 0. \quad (8)$$

Radial momentum

$$\rho u \frac{\partial V}{\partial z} + \rho V^2 = -\Lambda + \frac{\partial}{\partial z} \left(\mu \frac{\partial V}{\partial z} \right). \quad (9)$$

Energy

$$\rho c_p u \frac{\partial T}{\partial z} = \frac{\partial}{\partial z} \left(\lambda \frac{\partial T}{\partial z} \right) - \sum_k j_k \frac{\partial h_k}{\partial z} - \sum_k h_k W_k \dot{\omega}_k. \quad (10)$$

Species

$$\rho u \frac{\partial Y_k}{\partial z} = -\frac{\partial j_k}{\partial z} + W_k \dot{\omega}_k. \quad (11)$$

The variables appearing in Equations 8–11 are defined as follows:

- z is the axial coordinate;
- r is the radial coordinate;
- ρ is the density;
- u is the axial velocity;
- v is the radial velocity;
- $V = v/r$ is the scaled radial velocity;
- Λ is the pressure eigenvalue, which is independent of z ;
- μ is the dynamic viscosity;
- c_p is the mixture heat capacity at constant pressure;
- T is the temperature;
- λ is the thermal conductivity;
- Y_k is the mass fraction of species k ;
- j_k is the diffusive mass flux of species k ;
- $c_{p,k}$ is the specific heat capacity of species k ;
- h_k is the specific enthalpy of species k ;
- W_k is the molecular weight of species k ; and
- $\dot{\omega}_k$ is the molar production rate of species k .

Individual thickening factors for species and thermal diffusion are applied as F_{sp} and F_{th} , respectively. A corresponding reduction in the reaction rate is applied by dividing reaction-rate terms by F_r . The resulting equations for energy and species are shown in Equations 12 and 13.

$$\rho c_p u \frac{\partial T}{\partial z} = F_{th} \frac{\partial}{\partial z} \left(\lambda \frac{\partial T}{\partial z} \right) - F_{sp} \sum_k j_k \frac{\partial h_k}{\partial z} - \sum_k h_k W_k \frac{\dot{\omega}_k}{F_r}. \quad (12)$$

$$\rho u \frac{\partial Y_k}{\partial z} = -F_{sp} \frac{\partial j_k}{\partial z} + W_k \frac{\dot{\omega}_k}{F_r}. \quad (13)$$

Listing 5 and 6 show the changes to the energy and species equations respectively in the `src/oneD/Flow1D.cpp` file.

Listing 5: Modifications to the thermal-conduction, species enthalpy-flux, and chemical source terms in `Flow1D::evalEnergy`.

```

1 - double flxk = 0.5*(m_flux(k, j-1)
2   + m_flux(k, j));
3 + double flxk = 0.5*(m_flux(k, j-1)*m_fsp
4   + m_flux(k, j)*m_fsp);
5
6 - sum += m_wdot(k, j)*m_hk(k, j);
7 + sum += m_wdot(k, j)*m_hk(k, j)/m_fr;
8
9 - rsd[index(c_offset_T, j)] =
10 -   -m_cp[j]*rho_u(x, j)*dTdz(x, j)
11 -   - conduction(x, j) - sum;
12 + rsd[index(c_offset_T, j)] =
13 +   -m_cp[j]*rho_u(x, j)*dTdz(x, j)
14 +   - conduction(x, j)*m_fth - sum;
```

Listing 6: Modifications to the diffusion and chemical source terms in `Flow1D::evalSpecies`.

```

1 - double diffus = 2*(m_flux(k, j) - m_flux(k, j-1))
2   / (z(j+1) - z(j-1));
3 + double diffus = 2*(m_fsp*m_flux(k, j)
4   - m_fsp*m_flux(k, j-1))
5   / (z(j+1) - z(j-1));
6
7 - rsd[index(c_offset_Y + k, j)] =
8   (m_wt[k]*m_wdot(k, j) - convec - diffus)
9   / m_rho[j]
10 -   - rdt*(Y(x, k, j) - Y_prev(k, j));
11 + rsd[index(c_offset_Y + k, j)] =
12   (m_wt[k]*m_wdot(k, j)/m_fr - convec - diffus)
13   / m_rho[j]
14 +   - rdt*(Y(x, k, j) - Y_prev(k, j));
```

A setter function which is exposed to the user through Cython allows for custom assignment of F_{sp} , F_{th} , F_r , rather than computing these values at the backend. The backend assignment is shown in Listing 7.

Listing 7: Backend assignment of the species, thermal, and reaction-rate factors.

```

1 void Flow1D::setThickenedFlame(
2   double fth,
3   double fsp,
4   double fr
5 ) {
6   this->m_fsp = fsp;
7   this->m_fth = fth;
8   this->m_fr = fr;
9 }
```

This was done for the following reasons:

- The computation of these values is not computationally expensive, and hence can be calculated at the Python frontend easily, and supplied to the C++ backend.
- It gives the user flexibility to explore alternative definitions and relations between F_{sp} , F_{th} , and F_r without having to recompile the full code again.
- Both the ATF and S-TF can be explored in the same script, and using the same methods in the backend. This can be done by setting $F_{sp} = F_{th} = F_r$ at the front end. Also, unthickened solutions can be explored and validated with the baseline Cantera fork by setting $F_{sp} = F_{th} = F_r = 1$.

The frontend variables in Python are set as shown in Listing 8.

Listing 8: Python frontend method used to set the thickened-flame factors.

```
1 flame.set_stretched_thickened_flame(fth,fsp,fr)
```

The assigned values can be checked using the method shown in Listing 9.

Listing 9: Python frontend method used to retrieve the thickened-flame factors.

```
1 Fth, Fsp, Fr = flame.get_stretched_thickened_flame()
```

3 Artificially Thickened Flame Model

The artificially thickened flame model [3] is intended to increase the flame thickness by a factor $\delta_L^F = F\delta_L^0$ while preserving the laminar flame speed s_L^0 . To validate the implementation, fixed thickening factors of $F = F_{sp} = F_{th} = F_r$ are applied to the flame. Thickening factors of $F = 1, 2, 5$ and 10 are tested on a freely propagating flame. This script is available at `samples/python/stretchedTF/atf_simplified_validation.ipynb`. The temperature profile and thermal flame thickness for each case are shown in Figure 1.

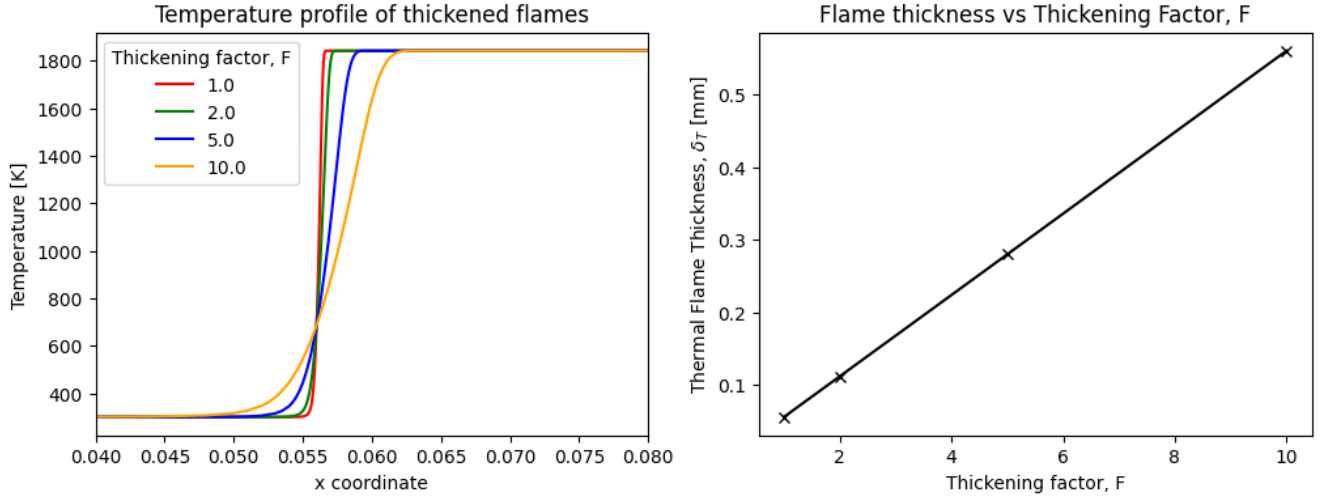


Figure 1: (Left) Temperature profile of each thickened flame. (Right) Thermal flame thickness for each thickening factor.

A comparison of s_L^0 and δ_L^F is shown in Table 2, where the maximum error is 0.2% in s_L^0 , and 0.6% in δ_L^F at $F = 10$.

4 Stretched Thickened Flame Model

The goal of the stretched thickened flame model is to control the stretched response of a particular flame, i.e., to conserve the consumption speed s_c vs κ relationship as thickening is applied. This can be done by describing the following relationship between F_{sp} , F_{th} , and F_r , as shown in Equation 14.

Table 2: Comparison of the freely propagating laminar flame properties obtained using various thickening factors. The flame thickness is defined using the maximum temperature-gradient method ($\delta_L^F = \delta_T$).

Thickening Factor	Laminar flame speed, s_L^0 m/s	Flame thickness, δ_L^F mm
F=1	0.2054	0.5596
F=2	0.2054	1.1193
F=5	0.2049	2.7988
F=10	0.2049	5.6011

$$\begin{cases} F_{th} = F \\ F_{sp} = \frac{F^2 Le^0}{F + (Le^0 - 1)\tilde{X}_0} \\ F_r = \left[\frac{1}{F_{th}^{1/2}} \left(\frac{F_{th}}{F_{sp}} \right)^{\beta_{fuel}} \right]^{-2} \end{cases}, \quad (14)$$

Here, Le_0 is the Lewis number of the unthickened reference flame, and β_{fuel} is a fuel-specific constant defined by Detomaso et al.[4]. The parameter $\tilde{X}_0$ controls the effective Lewis number of the thickened flame through the species-diffusion factor F_{sp} . The reaction-rate factor F_r is then calculated such that the unstretched laminar flame speed is preserved. Further details of the optimisation procedure are provided by Detomaso et al.[4].

In summary, $\tilde{X}_0$ is modified to adapt the consumption speed of the thickened flame to match that of an unthickened flame at a target stretch level, i.e. to minimise:

$$\varepsilon = \frac{|s_c(F=1) - s_c(F>1)|}{s_c(F=1)} \bigg|_{\kappa_s^{\text{target}}}. \quad (15)$$

This optimisation procedure is implemented in `samples/python/stretchedTF/stf_validation.ipynb`, and the resulting thickening factors are compared with published results from Detomaso et al. [4], as seen in Table 3. The results are largely consistent with the results of Detomaso et al., with the largest difference being at $F_{th} = 2$, where the difference in F_{sp} and F_r were -2.5% and 1.83% respectively. These differences may arise from differences in the Cantera version, solver configuration, and numerical tolerances.

Table 3: Comparison of the thickening factors obtained in the present implementation with those reported by Detomaso et al. The Detomaso reaction-rate factor is expressed using the reciprocal convention adopted in the present implementation.

F_{th}	F_{sp}	$F_{sp,Det}$	ΔF_{sp} (%)	F_r	$F_{r,Det}$	ΔF_r (%)
1	1.0000	1.0000	0.0000	1.0000	1.0000	0.0000
2	2.2951	2.3530	-2.4615	1.8024	1.7699	1.8342
5	6.5816	6.6690	-1.3113	4.0620	4.0161	1.1427
7	9.7038	9.9130	-2.1101	5.4684	5.3763	1.7129
10	14.6935	14.7060	-0.0849	7.4757	7.4627	0.1747

Therefore, the stretched thickened flame implementation is considered validated for the one-dimensional test cases examined in this document. Further validation should be performed before applying the implementation to new fuels, chemical mechanisms, configurations, or large-eddy simulations.

References

- [1] Benedetta Franzelli, Eleonore Riber, and Bénédicte Cuenot. Impact of the chemical description on a Large Eddy Simulation of a lean partially premixed swirled flame. *Comptes Rendus Mécanique*, 341(1):247–256, January 2013.
- [2] Cantera Developers. Governing equations for one-dimensional flow. <https://cantera.org/dev/reference/onedim/governing-equations.html>, 2026. Cantera 4.0.0a2 development documentation, accessed 1 July 2026.

- [3] O. Colin, F. Ducros, D. Veynante, and T. Poinso. A thickened flame model for large eddy simulations of turbulent premixed combustion. *Physics of Fluids*, 12(7):1843–1863, July 2000.
- [4] Nicola Detomaso, Jean-Jacques Hok, Omar Dounia, Davide Laera, and Thierry Poinso. A generalization of the Thickened Flame model for stretched flames. *Combustion and Flame*, 258:113080, December 2023.